DSS5105 CAPSTONE · TRACK 2 · SPRINT 1

# Subcontracting Desk

From informal chat decisions
to verified, accountable outsourcing dispatch

**Group NN**

屈妍玥 Qu Yanyue · 张锦若 Zhang Jinruo · 吴杰 Wu Jie
李岩 Li Yan · 吴若晗 Wu Ruohan · 许佩瑶 Xu Peiyao

github.com/dss5105-track2/track2-desk · 16 October 2026



# A batch goes outside on one chat message, unchecked and unrecorded

**TODAY**

*"@Chen — order 042, 1,500 crewnecks. Send it to QuickStitch, need it back by the 19th."*

1. **Can they make it?** Not every workshop makes every product.
2. **Can they take it now?** A fast shop with a 3-day queue is not fast today.
3. **Are they allowed to?** One shop is suspended after an audit; one is on trial, small batches only.

*Get one wrong and the batch bounces back a day later. In a ten-day cycle that is the margin gone.*

**The decision:** which eligible workshop, with capacity, returns the batch by its due date — and why.

**WHAT THE DESK MUST DO**

**Read the chat.** Extract the request; ask when it is ambiguous or incomplete; refuse, with a reason and an alternative, when the named workshop cannot take it; honour constraints and record who set them.

**Decide on numbers.** Keep the workshop register; estimate a delivery date per candidate before committing; allocate with the reasoning attached; when nobody can make the date, say so and lay out the options.

**Leave a record.** An auditable record of every decision — what the group chat never produced.

*Optional, assessed after Sprint 2: the stage extension (--stages). Not pursued: multi-agent negotiation — the mechanism is not graded.*

**BUSINESS VALUE**

- **Dispatcher:** faster decisions with verified recommendations
- **Factory:** fewer late deliveries, shown in simulation
- **Customer:** more reliable delivery-date commitments

# The model reads, code decides, the dispatcher confirms

INPUT

## Chat message

Free text from five people, as they type it.

LANGUAGE MODEL + CHECK

## Language layer

The model extracts order, pieces, date, constraint. Code checks each value against the text.

CODE · NO LLM

## Kernel

Register: make it, take it, allowed? Estimator: two dates per shop. Allocator: the objective is a setting.

CODE

## Explain and check

Writes the reason from tool outputs. Flags any number that is not in one.

HUMAN

## Dispatcher

Confirms, changes or escalates. Only then is capacity booked and the audit record written.

1

2

3

**1 Text decides.** Every value the model extracts is checked against the message; anything the message does not contain is dropped.

**2 Eligibility before ranking.** Three hard checks first; then every eligible shop gets two dates — expected, and if the batch is reworked — and the ranking follows the chosen objective.

**3 Traced, then booked.** Every number in the explanation must come from a tool output; capacity is booked and the audit record written only after the dispatcher confirms.

### ONE REAL MESSAGE, R12, THROUGH THE FIVE STAGES

| | | | | |
|---|---|---|---|---|
| *"ORD-053 — 800 polo shirts for UrbanThread, due Apr 10. Boss says nothing new to QuickStitch this week."* | ORD-053 · 800 pcs · due 10 Apr · constraint: QuickStitch excluded — standing, set by Boss | 4 shops eligible. GiantWeave back Fri 10 Apr, on time; Sat 11 Apr if reworked, a day late | *"GiantWeave: on time; it only makes tops, so the two shops that also make accessories stay free for tighter batches."* | Confirmed → ledger updated, audit record A0004 |

# Primary objective: minimise late deliveries

**Late-delivery rate** = batches returned after their due date ÷ all batches. Also reported: mean days late (0 when on time). Both are the simulator's own metrics.

## WHY THIS ONE

1. **Relevant.** The due date is the one constraint every order carries; quick response exists to protect it — "the factory that farms out a batch today beats the one that decides on Thursday."

2. **Measurable.** The simulator records when every batch returns, so lateness is computed, not estimated.

3. **Controllable.** Lateness depends on queue, capacity and transport — exactly what the choice of workshop changes.

## THE OTHER THREE

**Defects, fairness.** Switchable settings of the same allocator; results reported on the same 120 orders.

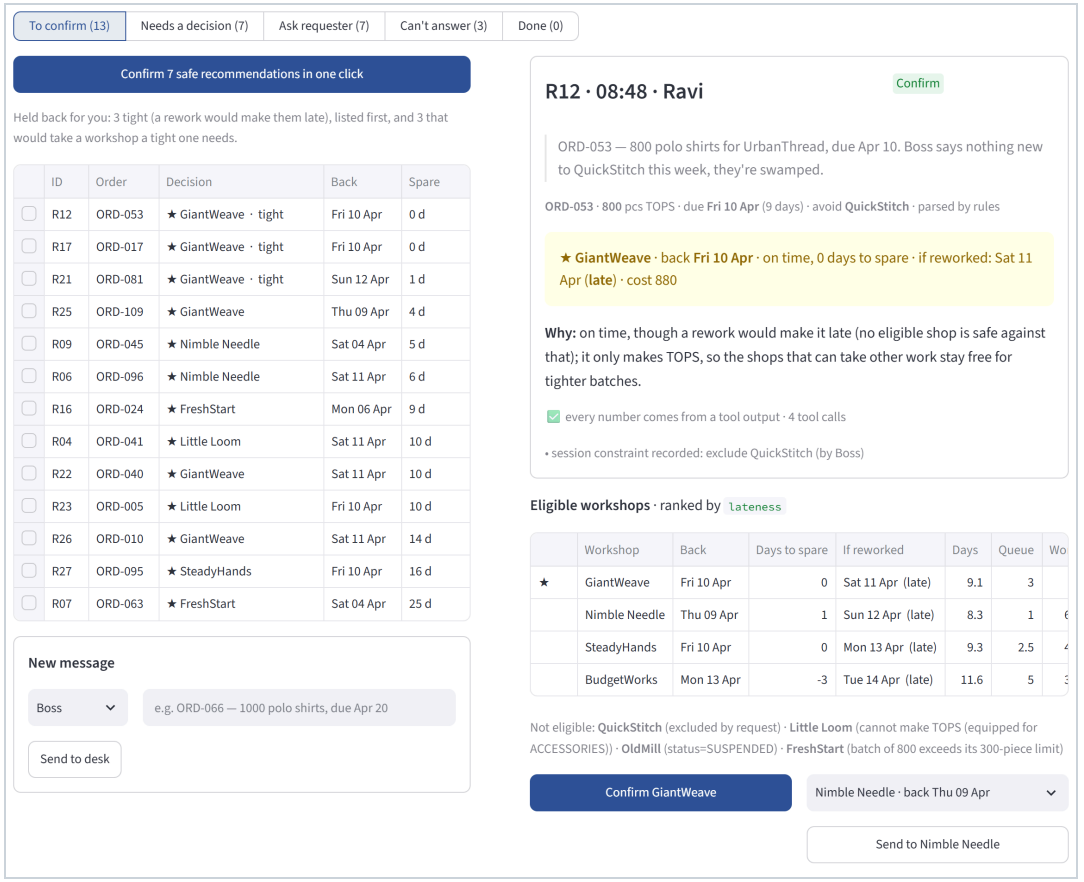
**Hybrid.** One weighted setting (late 1.0, finish 0.3, defect 0.25, load 0.5). Metrics scaled 0–1 first; weights at the knee of the lateness–defect curve, checked on held-out seeds.

**Robust to failure.** Tested with the shock scenario; guardrail: lateness no worse than random.

*The three baselines are 34–92% late in simulation: the largest gap, but supporting evidence, not the reason.*

Declared tie-break: when two settings are equally late, we take the shorter turnaround — the track names turnaround days as the business impact.

# The loop runs end to end, and the first results are in




One message in, one recommendation with two dates and a reason, one click to confirm, one audit record.

**IN SIMULATION · OFFICIAL 120 ORDERS · MEAN OF 10 SEEDS**

| Policy | Late batches | Mean days late |
|---|---|---|
| random (baseline) | 34.2% | 6.13 |
| cheapest (baseline) | 91.8% | 81.9 |
| earliest-finish (simple rule, no LLM) | 1.7% | 0.18 |
| **ours (lateness setting)** | **1.7%** | **0.09** |

On 200 fresh order books the simple rule never saw, ours is ahead: 0.9% vs 1.7% late on 100 jittered sets (better on 72, worse on 3). Under shock the simple rule still edges us, 1.9% vs 2.0%: one batch in ten runs. Source: simulate.py v3, commit 1c6cb61.

- **56 automated tests pass;** every number in every reply on the 30-message inbox traces to a tool output.
- **All six of us labelled the 30 messages independently** (κ 0.70); labels frozen on 7 Oct; counts per kind match the instructor's mix.
- **The language model was measured, then corrected:** GPT-5 mini now reads all 30 official and 38 new messages right; the cheapest model started at 86%.

# What could go wrong, and what we are assuming

| RISK | IMPACT | MITIGATION |
| --- | --- | --- |
| **No clean language test yet.** The 30 official messages were used in development; the 38 new ones are AI-written. | We may overstate how well the model reads. | Held-out inbox: 44 messages written by four teammates who do not touch the prompt; answers frozen before the first run (frozen 14 Oct, first run 15 Oct). |
| **The simulator is kinder than reality.** Seven-day week; the estimator does not know when a workshop closes. | Lateness looks better than it would be. | Sunday variant reported separately (simple rule 1.7 → 3.7%, ours → 2.1%); closed-workshop detection in Sprint 2. |
| **Our online policy is not globally optimal.** A hindsight search that knows every order in advance reaches 0% late in the tested setting. | Room we have not captured. | A dispatcher cannot see future orders; we report the gap and keep the rule simple enough to explain. |

**Assumptions:** requests arrive in time order and a confirmed dispatch occupies capacity · a standing constraint ("nothing new to QuickStitch this week") applies to every later request · a reworked batch takes half as long again (simulator rule) · the data holds no customer prices and no history, so those questions are declined.

# Sprint 2 (30 Oct) and Sprint 3 (13 Nov): from a working loop to evidence we can defend

| WORK | ACCEPTANCE | OWNER | WHEN |
|---|---|---|---|
| **Held-out inbox** | 44 hand-written messages, answers frozen first, ≥ 90% right behaviour | 张锦若 freezes; 李岩, 吴若晗 run | written 9–11 Oct; frozen 14 Oct; first run 15 Oct |
| **Closed-workshop detection** | In the shock run the closed shop is detected and gets no new batches; shock lateness no worse than normal | 吴若晗 | design 14 Oct; done 27 Oct |
| **User test** | 2–3 students from other groups complete the 9 tasks | 李岩, 屈妍玥 | by 28 Oct |
| **Public demo** | Reachable from an outside browser; one dispatch completed | 李岩 (吴杰 logs in) | by 28 Oct |
| **Evaluation draft** | 10 failure cases with root causes | 李岩, 吴若晗 (causes); 吴杰, 许佩瑶 (text) | 24–29 Oct |
| **Optional: stage extension (--stages)** | Assessed after Sprint 2, if time allows | — | — |

SUCCESS CRITERIA · MEASURED ON v3 DATA

**≤ 2%**
late on the official orders, normal and shock (now 1.7% / 2.0%)

**Beat it**
fewer late batches than the simple rule on fresh order books (now 0.9% vs 1.7%)

**≥ 90%**
right behaviour on the held-out inbox, labels frozen first

**100%**
numbers traced; every “if reworked” date kept (now 100%; 99% in shock)

# Appendix A · Results on v3 data, mean of 10 seeds

| Policy | Late % | Late days | Mean days | Defective pcs % | Max share % | Cost | Late % in shock |
|---|---|---|---|---|---|---|---|
| random (official) | 34.2 | 6.13 | 21.9 | 5.6 | 23 | 111,780 | 38.2 |
| greedy_biggest (official) | 84.2 | 26.53 | 50.3 | 5.3 | 70 | 108,430 | 84.9 |
| cheapest (official) | 91.8 | 81.92 | 106.2 | 8.2 | 70 | 85,960 | 91.8 |
| **earliest_finish (simple rule)** | **1.7** | **0.18** | **8.3** | **7.2** | **26** | **115,566** | **1.9** |
| **lateness: specialist-first rule (ours)** | **1.7** | **0.09** | **9.3** | **7.7** | **28** | **115,438** | **2.0** |
| use_slack: latest safe finish (ours) | 1.0 | 0.07 | 14.0 | 6.8 | 32 | 105,502 | 1.7 |
| lateness, round 1 (ours) | 1.7 | 0.18 | 8.2 | 7.5 | 26 | 115,592 | 1.9 |
| hybrid (ours) | 4.3 | 0.29 | 10.1 | 6.1 | 29 | 111,238 | 5.5 |
| defects (ours) | 28.7 | 2.23 | 21.1 | 5.1 | 43 | 110,396 | 31.0 |
| fairness (ours) | 6.8 | 0.64 | 12.6 | 7.1 | 20 | 113,088 | 10.1 |
| cost (ours) | 4.4 | 0.24 | 17.2 | 5.7 | 38 | 100,931 | 5.2 |

simulate.py v3, commit 1c6cb61. use_slack is late less often and costs less, but takes 14 days on average. Single-seed defect draws move defective pcs by about ±2 points.

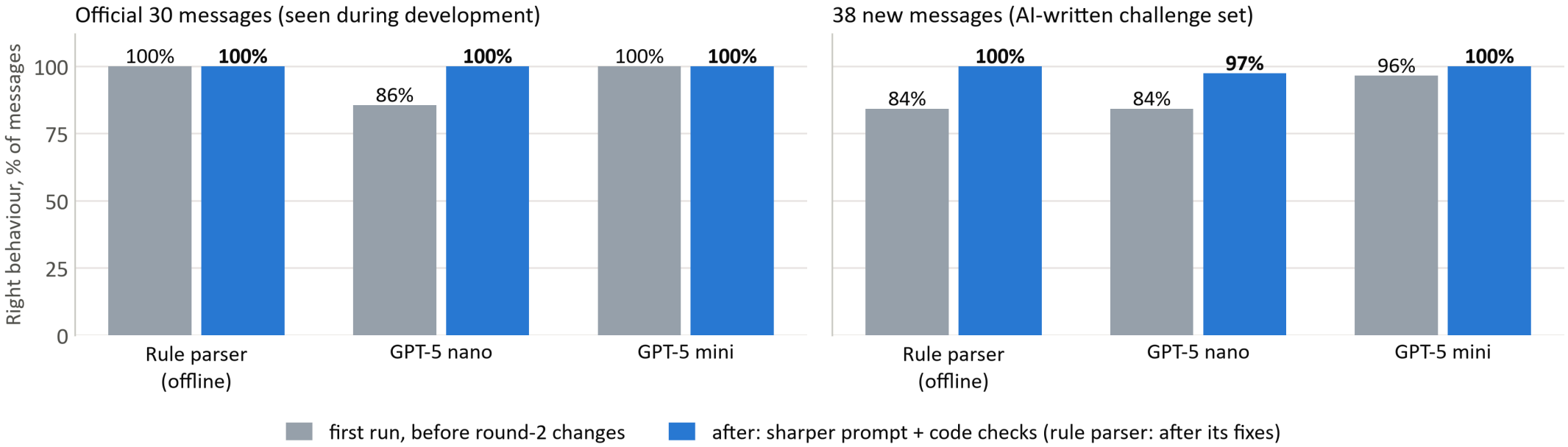
# Appendix B · How the desk sorted the 30 messages

| | | | | | |
|---|---|---|---|---|---|
| **R01** ask: missing info | **R02** can't answer | **R03** ask: which order | **R04 ★** allocate | **R05** ask: missing info | **R06 ★** allocate |
| **R07 ★** allocate | **R08** refuse | **R09** allocate | **R10** ask: missing info | **R11** refuse | **R12** allocate |
| **R13** can't answer | **R14** refuse | **R15** ask: which order | **R16 ★** allocate | **R17 ★** allocate | **R18** can't answer |
| **R19** ask: which order | **R20** no on-time option | **R21** allocate | **R22 ★** allocate | **R23 ★** allocate | **R24** refuse |
| **R25** allocate | **R26 ★** allocate | **R27 ★** allocate | **R28** no on-time option | **R29** ask: which order | **R30** no on-time option |

allocate · no on-time option · ask: which order · ask: missing info · refuse · can't answer

★ changed by the instructor in v3. Shown with the morning queues; the counts by kind match the instructor's published mix. Processed in order, 11 of the 13 on-time requests stay on time (R21 is 1 day late; R25 is 1 day late); at most 12 can.

# Appendix C · The language layer, measured on two inboxes




- **The rule parser was built around the 30:** 100% there, 84% on new wording at first try.
- **Model choice matters:** with the round-1 prompt GPT-5 nano gets 86% of the 30 right, GPT-5 mini 100%. After our checks nano still misreads one new phrasing (a date “sooner than planned, not said when”) in 3 runs of 3.
- **Not a clean test yet:** the 38 new messages are AI-written and two of our fixes came from errors seen on them. The held-out inbox will be written by teammates.

# Appendix D · Promised dates, and a six-day week

QUOTING TWO DATES · 1,200 BATCHES, LATENESS RULE

**94%**

of expected dates hold. Every miss in normal runs is a defect rework.

**100%**

of “if reworked” dates hold (99.2% with a workshop closed). They are 1.8 days later on average.

SUNDAYS OFF, AS THE PRIMER SAYS · LATE BATCHES %

| Policy | Seven-day week | Sundays off |
|---|---|---|
| random | 34.2 | 50.2 |
| earliest_finish (simple rule) | 1.7 | 3.7 |
| **lateness rule (ours)** | **1.7** | **2.1** |
| **lateness rule, estimate knows Sundays** | **1.7** | **1.8** |

Same orders, same defect draws; only change: no work on Sundays. The allocators are not told. With the Sunday rule switched off the variant reproduces simulate.py exactly.

# Appendix E · Our rule ties the simple rule on the official data, and wins when the data moves




**The rule, in order:** on time even if the batch is reworked; then the least flexible shop (a specialist before one that makes both categories); then earliest finish.

- **Why it helps:** earliest-finish spends the fast, flexible shops on batches that had slack; the next tight batch then has nowhere to go.
- **Official data:** a tie on late batches (1.7%), half the late days (0.09 against 0.18).
- **Fresh data:** better on 72 of 100 jittered order books, worse on 3.
- **Where the simple rule still wins:** mean turnaround, 8.3 against 9.3 days; the shock run, by one batch in ten.

Official: seeds 5105 to 5114. Fresh sets were generated after the rule was fixed (seeds 20000+), 3 seeds each. The allocators are not told about Sundays.

# Appendix F · Why not the setting with 1.0% late? Its edge does not hold on fresh data

Both settings share steps 1 and 2 (rework-safe shops, specialist first) and differ only in the tie-break: earliest finish, or the latest finish that is still safe (use_slack). Paired runs on the same data and seeds; a sign test on how often use_slack is later or earlier.

| Data | Runs | use_slack better / equal / worse on late % | Sign test p | Mean late % (ours / use_slack) | Mean turnaround, days |
|---|---|---|---|---|---|
| Official 120 orders, 10 seeds | 10 | 8 / 2 / 0 | 0.008 | 1.67 / 1.00 | 9.3 / 14.0 |
| Official, shock, 10 seeds | 10 | 5 / 4 / 1 | 0.219 | 2.00 / 1.67 | 9.4 / 14.0 |
| 100 jittered order books | 100 | 38 / 40 / 22 | 0.052 | 0.91 / 0.79 | 8.8 / 13.6 |
| 50 sets, tighter due dates | 50 | 26 / 4 / 20 | 0.461 | 4.52 / 4.37 | 8.0 / 10.0 |
| 50 sets, extra accessory orders | 50 | 23 / 11 / 16 | 0.337 | 2.75 / 2.69 | 9.8 / 14.5 |

**Reading.** use_slack is reliably better only on the official orders, which both settings were shaped on (8 of 10 seeds, $p = 0.008$). On 200 fresh order books the two are indistinguishable on lateness ($p \geq 0.05$). The robust differences are elsewhere: use_slack returns batches about 5 days later and costs about 9% less. The track names turnaround days as the business impact, so we declared the faster tie-break and keep use_slack as the cost-saving setting.